```
  | InitFromProps
  | SetTicketToBuy
  | SetTickets
  | UnholdTicket
): VenueState => {
```

We use the `type` keyword for the same reasons we'd name any variable: to give a complex operation a simpler, more semantically meaningful name, making the type easier to use and the code easier to read.

Now we can say that a variable is a member of a specific type, like `let s: HoldTicket`, or we can declare the variable to be a member of the union type, as in `let u: VenueAction`. These two declarations have different meanings and allow access to different attributes.

When we declare the variable as a member of the specific type, such as `let s: HoldTicket`, TypeScript lets the variable have access to all the attributes of `HoldTicket`.

When we say that a variable is a member of the union type, as in `let u: VenueAction`, that declaration in TypeScript only lets that variable have access to methods or attributes that are common to all the specific types that make up the union. In this case, we have seven different types in that union. All three of them have a `type` attribute, which means we can call `u.type`. As it happens, some of those types have a `seatNumber` attribute, but the rest don't, so if we call `u.seatNumber`, TypeScript will fail to compile.

All that said, if you look in our reducer code back in Chapter 11, *Managing State in React*, you'll see that we do declare the action as the generic type `Action`, and then inside the case statements, we do, in fact, call attributes like `action.dateString`, so there must be some way to tell TypeScript that you are looking at only one of the specific types of the union type. TypeScript actually has a few different ways of differentiating between types. We'll talk about the specific one used in our reducer code in *Using Enums and Literal Types*.

Union Types and Type Guards

TypeScript calls the functionality that lets TypeScript know it can use a specific